FULL STACK DEVELOPMENT

DOCUMENTATION

FlightFinder

Flight Booking Application

MERN Stack Project

Submitted By:
[Veera Chaitanya]
[23225A4507]
[CSE(Artificial Intelligence and Data Science)]

Academic Year: 2025-2026

📚 Full Stack Development Documentation

FlightFinder - MERN Stack Implementation

---

1. Introduction to Full Stack Development

Full Stack Development refers to the development of both frontend (client-side) and backend (server-side) portions of a web application. A full stack developer works with all layers of the application stack.

1.1 Technology Stack Overview

Layer	Technology	Purpose
Frontend	React.js	User Interface & Client-side Logic
Backend	Node.js + Express.js	Server-side Logic & API
Database	MongoDB	Data Storage & Management
Runtime	Node.js	JavaScript Runtime Environment

2. MERN Stack Architecture

Three-Tier Architecture

```
┌──────────────────────────────────────┐
│     PRESENTATION LAYER (Frontend)    │
│                                      │
│  React.js + Bootstrap 5 + Context API  │
│           Port: 3000                 │
└──────────────────────────────────────┘
                              ↕
┌──────────────────────────────────────┐
│    APPLICATION LAYER (Backend)       │
│                                      │
│    Node.js + Express.js + RESTful API  │
│           Port: 6001                 │
└──────────────────────────────────────┘
                              ↕
┌──────────────────────────────────────┐
│       DATA LAYER (Database)          │
│                                      │
│      MongoDB + Mongoose ODM          │
│           Port: 27017                │
└──────────────────────────────────────┘
```

2.1 Component Breakdown

- **M - MongoDB:** NoSQL database for flexible data storage
- **E - Express.js:** Web application framework for Node.js
- **R - React.js:** Frontend library for building user interfaces
- **N - Node.js:** JavaScript runtime for server-side execution

3. Frontend Development (React.js)

3.1 Project Structure

```
client/ ├── public/ │ ├── index.html │ └── manifest.json ├── src/ │ ├── components/
│ │ ├── Register.jsx │ │ ├── Login.jsx │ │ └── Navbar.jsx │ ├── pages/ │ │ ├──
LandingPage.jsx │ │ ├── BookFlight.jsx │ │ ├── CustomerDashboard.jsx │ │ ├──
OperatorDashboard.jsx │ │ └── AdminDashboard.jsx │ ├── context/ │ │ └──
GeneralContext.jsx │ ├── App.js │ └── index.js └── package.json
```

3.2 Key React Concepts Used

Concept	Implementation	Purpose
Components	Functional Components	Reusable UI building blocks
Hooks	useState, useEffect, useContext	State management & side effects
Context API	GeneralContext	Global state management
Routing	React Router v6	Client-side navigation
HTTP Client	Axios	API communication

3.3 Sample Component Code

```
// BookFlight.jsx - Flight Booking Component import React, { useState, useContext }
from 'react'; import { GeneralContext } from '../context/GeneralContext'; import
axios from 'axios'; function BookFlight() { const [journeyDate, setJourneyDate] =
useState(''); const [seatClass, setSeatClass] = useState('Economy'); const { user }
= useContext(GeneralContext); const handleBooking = async () => { const response =
await axios.post('http://localhost:6001/book-ticket', { userId: user._id,
journeyDate, seatClass }); console.log(response.data); }; return ( <div
className="container"> <input type="date" min={new Date().toISOString().split('T')
[0]} onChange={(e) => setJourneyDate(e.target.value)} /> <button onClick=
{handleBooking}>Book Now</button> </div> ); }
```

4. Backend Development (Node.js + Express.js)

4.1 Project Structure

```
server/ ├── models/ │ ├── User.js │ ├── Flight.js │ └── Booking.js ├── index.js └── package.json
```

4.2 Express.js Server Setup

```
// index.js - Express Server const express = require('express'); const mongoose = require('mongoose'); const cors = require('cors'); const bcrypt = require('bcrypt'); const app = express(); // Middleware app.use(express.json()); app.use(cors()); // MongoDB Connection mongoose.connect('mongodb://localhost:27017/flightfinder', { useNewUrlParser: true, useUnifiedTopology: true }); // Routes app.post('/register', async (req, res) => { const { username, email, password, usertype } = req.body; const hashedPassword = await bcrypt.hash(password, 10); const user = new User({ username, email, password: hashedPassword, usertype }); await user.save(); res.json({ message: 'Registration successful' }); }); app.listen(6001, () => { console.log('Server running on port 6001'); });
```

4.3 RESTful API Endpoints

HTTP Method	Endpoint	Description	Request Body
POST	/register	User registration	username, email, password, usertype
POST	/login	User authentication	email, password
POST	/add-Flight	Add new flight	flightName, flightId, origin, destination
GET	/fetch-flights	Get all flights	None
POST	/book-ticket	Create booking	user, flight, passengers, journeyDate
PUT	/cancel-ticket/:id	Cancel booking	bookingStatus: 'cancelled'

5. Database Design (MongoDB)

5.1 MongoDB Overview

MongoDB is a NoSQL document-oriented database that stores data in flexible, JSON-like documents.

> **Why MongoDB for FlightFinder?**
>
> - Flexible schema for evolving requirements
> - Easy integration with Node.js using Mongoose
> - Scalable for growing data
> - Fast read/write operations

5.2 Mongoose Schema Definitions

User Schema

```
// models/User.js const mongoose = require('mongoose'); const userSchema = new
mongoose.Schema({ username: { type: String, required: true }, email: { type: String,
required: true, unique: true }, usertype: { type: String, enum: ['customer',
'operator', 'admin'], required: true }, password: { type: String, required: true },
approval: { type: String, default: 'pending' } }, { timestamps: true });
module.exports = mongoose.model('User', userSchema);
```

Flight Schema

```
// models/Flight.js const flightSchema = new mongoose.Schema({ flightName: String,
flightId: { type: String, unique: true }, origin: String, destination: String,
departureTime: String, arrivalTime: String, basePrice: Number, totalSeats: Number,
operator: { type: mongoose.Schema.Types.ObjectId, ref: 'User' } }, { timestamps:
true }); module.exports = mongoose.model('Flight', flightSchema);
```

Booking Schema

```
// models/Booking.js const bookingSchema = new mongoose.Schema({ user: { type:
mongoose.Schema.Types.ObjectId, ref: 'User', required: true }, flight: { type:
mongoose.Schema.Types.ObjectId, ref: 'Flight', required: true }, passengers: [{
name: String, age: Number }], totalPrice: Number, journeyDate: Date, seatClass: {
type: String, enum: ['Economy', 'Premium Economy', 'Business', 'First Class'] },
bookingStatus: { type: String, default: 'confirmed' } }, { timestamps: true });
module.exports = mongoose.model('Booking', bookingSchema);
```

6. Authentication & Security

6.1 Password Hashing with bcrypt

```
const bcrypt = require('bcrypt'); // Registration - Hash password const saltRounds =
10; const hashedPassword = await bcrypt.hash(plainPassword, saltRounds); // Login -
Compare passwords const isMatch = await bcrypt.compare(plainPassword,
hashedPassword); if (isMatch) { // Login successful }
```

6.2 Security Features Implemented

Feature	Implementation	Purpose
Password Encryption	bcrypt with salt rounds	Secure password storage
CORS	cors middleware	Control cross-origin requests
Input Validation	Client & server-side	Prevent invalid data
Date Validation	min attribute on date inputs	Prevent past date bookings
Role-Based Access	User type checking	Restrict unauthorized access

7. State Management (React Context API)

7.1 Context Implementation

```
// context/GeneralContext.jsx import React, { createContext, useState } from
'react'; export const GeneralContext = createContext(); export const GeneralProvider
= ({ children }) => { const [user, setUser] = useState(null); const [isLoggedIn,
setIsLoggedIn] = useState(false); const login = (userData) => { setUser(userData);
setIsLoggedIn(true); localStorage.setItem('user', JSON.stringify(userData)); };
const logout = () => { setUser(null); setIsLoggedIn(false);
localStorage.removeItem('user'); }; return ( <GeneralContext.Provider value={{ user,
isLoggedIn, login, logout }}> {children} </GeneralContext.Provider> ); };
```

7.2 Using Context in Components

```
import { useContext } from 'react'; import { GeneralContext } from
'../context/GeneralContext'; function Dashboard() { const { user, isLoggedIn, logout
} = useContext(GeneralContext); if (!isLoggedIn) { return <Navigate to="/login" />;
} return ( <div> <h1>Welcome, {user.username}</h1> <button onClick=
{logout}>Logout</button> </div> ); }
```

8. API Integration with Axios

8.1 Axios Configuration

```
import axios from 'axios'; const API_BASE_URL = 'http://localhost:6001'; // GET
Request const fetchFlights = async () => { try { const response = await
axios.get(`${API_BASE_URL}/fetch-flights`); return response.data; } catch (error) {
console.error('Error fetching flights:', error); } }; // POST Request const
bookTicket = async (bookingData) => { try { const response = await axios.post(
`${API_BASE_URL}/book-ticket`, bookingData ); return response.data; } catch (error)
{ console.error('Error booking ticket:', error); } }; // PUT Request const
cancelBooking = async (bookingId) => { try { const response = await axios.put(
`${API_BASE_URL}/cancel-ticket/${bookingId}`, { bookingStatus: 'cancelled' } );
return response.data; } catch (error) { console.error('Error cancelling booking:',
error); } };
```

9. Routing (React Router)

9.1 Route Configuration

```
// App.js import { BrowserRouter, Routes, Route } from 'react-router-dom'; function App() { return ( <BrowserRouter> <Routes> <Route path="/" element={<LandingPage />} /> <Route path="/register" element={<Register />} /> <Route path="/login" element={<Login />} /> <Route path="/customer-dashboard" element={<CustomerDashboard />} /> <Route path="/operator-dashboard" element={<OperatorDashboard />} /> <Route path="/admin-dashboard" element={<AdminDashboard />} /> <Route path="/book-flight/:id" element={<BookFlight />} /> </Routes> </BrowserRouter> ); }
```

10. Deployment & Installation

10.1 Prerequisites

- Node.js (v14 or higher)
- MongoDB (v4.4 or higher)
- npm or yarn package manager

10.2 Installation Steps

```
# Clone repository git clone <repository-url> # Install server dependencies cd server npm install # Install client dependencies cd ../client npm install # Start MongoDB net start MongoDB # Start server (Terminal 1) cd server npm start # Start client (Terminal 2) cd client npm start
```

10.3 Environment Configuration

Component	Port	URL
React Frontend	3000	http://localhost:3000
Express Backend	6001	http://localhost:6001
MongoDB	27017	mongodb://localhost:27017

11. Key Features Implementation

11.1 Dynamic Pricing Logic

```
const calculatePrice = (basePrice, seatClass, passengerCount) => { const
classMultipliers = { 'Economy': 1.0, 'Premium Economy': 1.5, 'Business': 2.5, 'First
Class': 4.0 }; const multiplier = classMultipliers[seatClass]; return basePrice *
multiplier * passengerCount; };
```

11.2 Date Validation

```
// Prevent past date selection <input type="date" min={new
Date().toISOString().split('T')[0]} onChange={(e) => setJourneyDate(e.target.value)}
/>
```

11.3 Operator Approval Workflow

```
// Admin approves operator app.post('/approve-operator', async (req, res) => { const
{ userId } = req.body; await User.findByIdAndUpdate(userId, { approval: 'approved'
}); res.json({ message: 'Operator approved' }); });
```

12. Best Practices Followed

Category	Practice	Benefit
Code Organization	Component-based architecture	Reusability & maintainability
State Management	Context API for global state	Avoid prop drilling
API Design	RESTful conventions	Standardized endpoints
Security	Password hashing, validation	Data protection
Error Handling	Try-catch blocks	Graceful error management
Responsive Design	Bootstrap 5 grid system	Multi-device compatibility

13. Conclusion

FlightFinder demonstrates a complete full-stack web application using the MERN stack. The project successfully implements:

- Modern React.js frontend with hooks and Context API
- Robust Node.js/Express.js backend with RESTful APIs
- MongoDB database with Mongoose ODM
- Secure authentication with bcrypt
- Role-based access control
- Responsive UI design

> **Learning Outcomes:**
>
> - ✅ Full-stack development with MERN stack
> - ✅ RESTful API design and implementation
> - ✅ Database modeling with MongoDB
> - ✅ React component architecture
> - ✅ Authentication and security practices
> - ✅ State management with Context API